文献综述

面向 ROS 2 架构的 AI Agent 机器人任务调度系统设计与实现

摘要：本文围绕"面向 ROS 2 架构的 AI Agent 机器人任务调度系统"这一课题，从 ROS 2 系统架构与通信机制、机器人任务调度方法、LLM 工具调用范式演进与 Agent Loop 技术、面向机器人的 LLM 任务规划、3D 场景图与全局状态管理、VLA 技术与具身智能前沿、外部 AI 模型 API 集成七个方面，对国内外相关研究文献进行了系统的梳理与分析。通过文献调研，总结了各领域的研究现状、关键技术和发展趋势，明确了本课题的研究基础与技术路线，为后续系统设计与实现提供了理论依据和技术参考。

关键词：ROS 2；AI Agent；任务调度；大语言模型；Function Calling；3D 场景图；VLA；具身智能

1 引言

随着人工智能和机器人技术的快速发展，智能机器人系统正从传统的预编程执行模式向具备自主理解、规划与决策能力的方向演进。ROS 2 作为新一代机器人操作系统，为机器人系统开发提供了先进的通信机制和系统架构。与此同时，以大语言模型（LLM）为核心的 AI Agent 技术在复杂任务理解与自主决策方面展现出强大的能力。如何将 AI Agent 技术与 ROS 2 机器人系统深度融合，实现基于自然语言的智能任务调度，是当前机器人领域的重要研究方向。

2022 年以来，具身智能（Embodied AI）成为人工智能领域的核心方向之一。Google 的 SayCan[15]率先证明了 LLM 可以作为机器人的"大脑"，将自然语言指令转化为可执行的动作序列。此后，3D 场景图环境表征[19]、EmbodiedRAG 检索增强规划[20]、VLA（Vision-Language-Action）端到端控制[27]等技术相继涌现，推动了机器人智能化水平的快速提升。然而，这些前沿研究大多针对特定硬件平台设计，尚未形成与 ROS 2 通信机制深度集成的通用方案。

本文献综述围绕本课题的核心技术，从以下七个方面展开：（1）ROS 2 系统架构与通信机制；（2）机器人任务调度方法；（3）LLM 工具调用范式演进与 Agent Loop 技术；（4）面向机器人的 LLM 任务规划；（5）3D 场景图与全局状态管理；（6）VLA 技术与具身智能前沿；（7）外部 AI 模型 API 集成技术。通过对相关文献的系统梳理，明确本课题的研究基础、技术挑战和解决思路。

2 ROS 2 系统架构与通信机制

2.1 ROS 的发展历程

Quigley 等人[1]于 2009 年提出了 ROS（Robot Operating System）的设计理念和系统架构。ROS 作为一个开源的机器人软件框架，提供了硬件抽象、设备驱动、消息传递、包管理等功能，极大地促进了机器人软件的开发效率和代码复用。ROS 1 采用基于 TCP/UDP 的自定义通信协议，通过中心化的主节点（roscore）实现节点发现和通信管理。

然而，ROS 1 的中心化架构和通信机制在实时性、可靠性和安全性方面存在明显不足。Maruyama 等人[2]对 ROS 2 的通信性能进行了系统评估，指出 ROS 2 基于 DDS（Data Distribution Service）的通信机制在延迟、吞吐量和可靠性方面均优于 ROS 1，特别是在多节点、高频率通信场景下表现更为突出。

2.2 ROS 2 的架构设计

Macenski 等人[3]全面介绍了 ROS 2 的设计理念、系统架构和实际应用。ROS 2 采用分层架构设计，底层基于 DDS 通信中间件，通过 RMW（ROS Middleware）抽象层实现与具体 DDS 实现的解耦。ROS 2 提供了三种核心通信机制：

- **话题（Topic）**：基于发布-订阅模式的异步通信，适用于传感器数据等连续数据流的传输；
- **服务（Service）**：基于请求-响应模式的同步通信，适用于短时间内可完成的查询操作；
- **动作（Action）**：面向长时间运行任务的通信机制，支持目标发送、过程反馈和结果获取，适用于导航、运动规划等任务。

Koubaa[4]在其编著的 ROS 参考手册中，系统总结了 ROS 2 在不同应用场景中的使用方法和最佳实践。值得注意的是，ROS 2 Action 接口的"目标-反馈-结果"三阶段模型与 AI Agent 的"指令-执行-回报"交互模式具有天然的对应关系，这为两者的深度集成提供了架构基础。

2.3 小结

ROS 2 通过引入 DDS 通信中间件、去中心化架构和服务质量（QoS）策略，有效解决了 ROS 1 在实时性、可靠性和安全性方面的不足。其 Topic、Service 和 Action 三种通信机制为机器人系统的模块化开发提供了灵活的接口。本课题将充分利用 ROS 2 的 Action 通信机制，实现 AI Agent 与机器人执行层的高效对接，并设计通信适配层隔离 ROS 2 版本差异。

3 机器人任务调度方法

3.1 基于行为树的任务调度

行为树（Behavior Tree）是当前机器人任务调度领域最广泛使用的方法之一。Colledanchise 和 Ögren[5]系统介绍了行为树在机器人和人工智能中的应用，指出行为树相比有限状态机具有更好的模块化特性和可扩展性。行为树通

过树状结构组织任务节点，支持顺序执行、并行执行、条件判断和装饰器等控制流，能够表达复杂的任务逻辑。

Macenski 等人[6]在 Navigation2 框架中采用行为树作为任务调度引擎，实现了移动机器人的自主导航。Navigation2 使用 BehaviorTree.CPP 库，通过 XML 配置文件定义导航行为的执行逻辑，支持路径规划、轨迹跟踪、障碍物避让和恢复行为等功能的灵活组合。

然而，行为树在实际应用中存在以下不足：（1）配置复杂度高——每新增一个任务场景，开发者需手动编写 BT XML 节点、定义条件判断逻辑，一个包含导航、巡逻、异常处理的复合任务可能需要编写包含十余个节点的行为树文件；（2）复用性差——行为树与具体任务场景紧耦合，为"巡逻"编写的行为树无法直接复用于"取物配送"场景；（3）使用门槛高——行为树的编写和调试需要具备 ROS 2 开发经验，非专业操作人员完全无法参与机器人任务的定义与调整。

3.2 基于有限状态机的任务调度

有限状态机（Finite State Machine, FSM）是另一种经典的任务调度方法。FSM 通过定义有限个状态和状态之间的转移条件来描述任务的执行流程。FlexBE 是 ROS 生态中基于分层状态机的行为引擎，提供了可视化的状态机编辑工具和在线监控功能。

然而，FSM 在处理复杂任务时存在状态爆炸问题，即随着任务复杂度的增加，状态数量和转移条件呈指数级增长，导致系统难以维护和扩展。

3.3 基于强化学习的任务调度

Sutton 和 Barto[7]在其经典著作中系统介绍了强化学习的理论基础和算法。强化学习通过智能体与环境的交互，学习最优的行为策略，在机器人任务调度领域具有广阔的应用前景。然而，强化学习方法通常需要大量的训练数据和计算资源，且在实际机器人系统中的泛化能力有限，难以直接应用于开放环境下的通用任务调度。

3.4 基于 AI Agent 的任务调度

近年来，基于 AI Agent 的任务调度方法逐渐成为研究热点。Li 和 Xiao[8]探讨了将 AI 与机器人技术集成用于自主任务调度和执行的方法，提出了一种基于智能体的任务分配和协调框架。该研究表明，AI Agent 能够有效提升机器人系统在动态环境中的任务调度效率和适应性。

Russell 和 Norvig[9]在《人工智能：一种现代方法》中系统阐述了智能体的理论基础，包括理性智能体、搜索算法、规划方法和决策理论等，为 AI Agent 的设计提供了重要的理论指导。

Zeng 等人[25]提出的 H-AIM（2026）将 LLM、PDDL 规划器和行为树进行层级化编排，用于多机器人长时域规划。LLM 负责语义理解和初步分解，PDDL 等形式化方法负责精确规划和验证，行为树负责底层执行。这一工作展示了 AI Agent 与传统调度方法融合的可能性。

3.5 小结

传统的行为树和有限状态机方法虽然在结构化任务中表现良好，但存在配置复杂、复用性差和使用门槛高等问题。基于 AI Agent 的任务调度方法能够利用自然语言理解和自主规划能力，有效降低系统配置的复杂度。本课题将采用 AI Agent 方法替代传统的行为树/状态机调度方式，实现更加灵活和智能的任务调度。

4 LLM 工具调用范式演进与 Agent Loop 技术

4.1 从文本解析到 Function Calling

LLM 与外部工具的交互方式经历了显著的技术演进。早期方案（2022 年以前）通过 Prompt Engineering 让 LLM 输出特定格式的文本（如 JSON），再由外部程序解析执行，这种方式依赖脆弱的正则匹配，格式错误率高。

2023 年 6 月，OpenAI 在 GPT 系列模型中引入 Function Calling 机制[10]。开发者预先定义函数的名称、参数 schema 和自然语言描述，LLM 在推理时可选择调用合适的函数并输出结构化的参数 JSON。Function Calling 的核心特征包括：单轮决策（LLM 在一次推理中决定调用哪个函数及其参数）、开发者执行（LLM 仅输出调用意图，实际执行由开发者代码完成）、结果回传（函数执行结果作为新消息回传给 LLM）。这一机制首次实现了 LLM 与外部工具的可靠结构化交互，为 Agent 系统的工具调用提供了标准化接口。

4.2 ReAct 与 Agent Loop

Yao 等人[11]于 2022 年提出 ReAct（Reasoning + Acting）框架，将 LLM 的推理能力与工具调用行为交织在一个循环中。其核心模式为：Thought（思考）→ Action（调用工具）→ Observation（观察结果）→ Thought（再思考）→ ... → Final Answer（最终回答）。

ReAct 的关键突破在于引入了 Agent Loop（智能体循环）：LLM 不再是"一次性输出答案"，而是在一个循环中持续推理、调用工具、观察结果、调整策略，直到任务完成。Function Calling 在此框架中成为 Agent Loop 内部的"执行动作"环节。LangChain、AutoGPT 等框架在 2023-2024 年间将 ReAct 模式工程化，使其成为构建 AI Agent 的主流范式。

4.3 平台级 Agent 基础设施

2025 年，Agent 技术从框架层面上升到平台基础设施层面。OpenAI 发布 Responses API 和 Agents SDK，支持在单次 API 调用中编排多个工具，内置 Agent Loop、Handoffs（Agent 间任务委托）和 Guardrails（安全护栏）。

同期，两个重要的开放协议标准出现：Anthropic 于 2024 年 11 月发布 MCP（Model Context Protocol），解决"工具如何被发现和连接"的问题；Google 于 2025 年 4 月发布 A2A（Agent-to-Agent Protocol），解决"不同 Agent 之间如何通信协作"的问题。这些协议标准为未来多机器人 Agent 协作提供了通信基础。

4.4 技术演进的层次关系

需要强调的是，上述四个阶段并非简单的替代关系，而是层次递进的包含关系：LLM 推理是最内层的基础能力；Function Calling 在其上提供结构化工具调用；Agent Loop 在 Function Calling 之上实现多轮自主推理；平台级 Agent 基础设施在最外层提供工程化支撑。每一层都建立在下一层之上。

4.5 小结

LLM 工具调用范式从文本解析到 Function Calling 再到 Agent Loop 的演进，为构建智能机器人任务调度系统提供了日益成熟的技术基础。Function Calling 的"函数名 + 参数"模式与 ROS 2 Action 的"Goal 类型 + 参数"模式天然匹配，这一对应关系是本课题技术路线的核心依据。本课题 V1 阶段采用 Function Calling 实现开环规划，架构中预留 Agent Loop 的扩展点以支持 V2 阶段的闭环规划升级。

5 面向机器人的 LLM 任务规划

5.1 开环规划：SayCan 与 Code as Policies

将 Agent Loop 的概念映射到机器人任务规划领域，对应的是"开环规划"与"闭环规划"的区别。

Ahn 等人[15]提出的 SayCan（Google, 2022）是开环规划的代表性工作。SayCan 的核心贡献是提出了"将语言理解锚定在机器人可用能力上"（grounding language in robotic affordances）的思想：LLM 提供任务理解（"我应该做什么"），每个机器人技能的价值函数提供可行性评估（"我能做什么"），两者相乘选出最优动作。然而，SayCan 是开环的——一旦选定动作序列就按序执行，如果某个技能执行失败，系统无法自主调整计划。此外，每个技能需要单独训练价值函数，扩展成本高。

Liang 等人[12]提出的 Code as Policies（2022）利用 LLM 直接生成机器人控制代码，将自然语言指令转化为可执行的 Python 程序。这种方法避免了传统任务规划中的符号化表示，提供了更加灵活的任务定义方式，但同样属于开环模式。

5.2 闭环规划：Inner Monologue 与 BrainBody-LLM

Huang 等人[16]提出的 Inner Monologue（Google, 2022）在 SayCan 的基础上引入了闭环反馈机制。系统在执行过程中持续将多种环境反馈注入 LLM 的规划 prompt 中，形成"内心独白"：成功检测反馈（"抓取动作失败，物体未被拾起"）、场景描述反馈（"当前场景中可见：桌子上有红色杯子和蓝色盘子"）、人类交互反馈（"用户说：不是那个，是旁边的杯子"）。LLM 根据这些反馈动态调整后续计划，实现了机器人领域的 Agent Loop。实验表明，闭环反馈显著提高了长时任务的完成率。

Dalal 等人[17]提出的 BrainBody-LLM（2024）借鉴人类神经系统的分层结构，将规划分为"大脑"（高层 LLM 负责任务分解）和"身体"（低层 LLM 负责动作生成）两层，并通过仿真器反馈实现闭环纠错。当低层执行失败时，错误信息反馈给高层 LLM 进行重新规划。

Driess 等人[18]提出的 PaLM-E（Google, ICML 2023）将视觉、语言和机器人状态统一编码到一个 562B 参数的多模态大模型中，实现了从感知到规划的端到端推理。虽然 PaLM-E 展示了强大的能力，但其巨大的模型规模使其难以在实际机器人系统中部署。

5.3 反幻觉与规划验证

LLM 在生成机器人任务计划时可能产生幻觉——当任务在物理上不可行时，LLM 仍可能生成看似合理但实际不可执行的规划序列。

Wan 等人[23]提出的 EmbodiedAgent（2025）聚焦于异构多机器人系统的幻觉问题，将不可行场景归纳为四类错误信号：LoA（缺乏能力）、LoS（缺乏技能）、LoL（超出负载）、LoO（缺乏物体）。通过在 MultiPlan+ 数据集（18000+ 标注实例）上对 Llama-3.1-8B 进行监督微调，使 LLM 学习在遇到不可行场景时主动调用错误信号函数。实验表明，经过领域自适应微调的 8B 参数模型显著优于未微调的 70B+ 大模型，证明了领域特定微调优于朴素参数扩展。

Ekpo 等人[24]提出的 VeriGraph（2024）通过场景图约束检查验证 LLM 生成计划的可执行性，包括空间约束验证和状态约束验证。与 EmbodiedAgent 的微调方案相比，场景图验证方案在泛化性和可解释性上具有优势——微调方案的错误检测能力受限于训练数据分布，而场景图验证基于实际环境状态，对任何可观测的约束违反都能检测。

5.4 多机器人协作规划

Chen 等人[26]提出的 EMOS（ICLR 2025）面向异构多机器人协作，每个机器人 Agent 通过理解自身 URDF 文件自动生成"能力简历"，基于物理能力约束进行规划。Zeng 等人[25]提出的 H-AIM（2026）将 LLM、PDDL 规划器和行为树进行层级化编排，LLM 负责语义理解，PDDL 负责形式化验证，行为树负责底层执行。

5.5 开环与闭环的对比分析

维度	开环规划（SayCan）	闭环规划（Inner Monologue）	Function Calling Agent
反馈机制	无	成功检测 + 场景描述 + 人类交互	工具返回结果
错误处理	预定义重试逻辑	LLM 分析失败原因后自主调整	LLM 观察错误后自主调整
扩展成本	高（每技能需训练价值函数）	高（需场景特定反馈模块）	极低（JSON schema 描述）
复合任务	LLM 一次性分解	动态分解，可根据中间结果调整	动态分解 + 工具组合

Function Calling 作为工具调用的底层机制，结合 Agent Loop 的闭环推理模式，在扩展成本上具有显著优势——新增能力只需定义 JSON schema 描述，无需训练价值函数或微调大模型。

5.6 小结

面向机器人的 LLM 任务规划已从开环模式（SayCan）演进到闭环模式（Inner Monologue、BrainBody-LLM），反幻觉研究（EmbodiedAgent、VeriGraph）为规划可靠性提供了保障。本课题借鉴 SayCan 的能力锚定思想，采用 Function Calling 实现更轻量的能力注册方案；借鉴 Inner Monologue 的闭环反馈思想，设计执行中反馈驱动的 LLM 重规划机制；借鉴 VeriGraph 的场景图验证思想，设计 SceneGraphValidator 对 LLM 生成的计划进行形式化验证。

6 3D 场景图与全局状态管理

6.1 问题背景

LLM 驱动的机器人任务规划系统面临一个核心矛盾：环境表征的丰富性与 LLM 上下文窗口的有限性之间的冲突。随着机器人操作环境从单房间扩展到多楼层建筑，3D 场景图中的实体数量可能从数十增长到数千。直接将完整场景图序列化为 LLM 输入会导致：token 数量超出模型上下文窗口限制、LLM 注意力偏差（Liu 等人[33]指出 LLM 对输入中间位置的信息关注度显著低于首尾位置，即"Lost in the Middle"现象）、任务无关信息干扰规划质量（Shi 等人[34]证实 LLM 易被无关上下文分散注意力）、以及规划延迟随 token 数量线性增长。

6.2 SayPlan：3D 场景图与层级语义搜索

Rana 等人[19]提出的 SayPlan（CoRL 2023）首次将 3D 场景图引入 LLM 规划。3D 场景图采用层级化语义环境表征（Building → Floor → Room → Object），节点包含语义标签和属性，边表示空间关系（contains / on / in / near / connected_to）。SayPlan 提出语义搜索机制：LLM 通过 `expand(node)` / `contract(node)` API 在层级化场景图上手动展开/折叠子图，逐步定位任务相关区域。

SayPlan 的局限性在于：压缩效果依赖场景图的层级结构；LLM 需要理解图结构并执行多轮展开/折叠操作，增加规划步骤和 token 消耗；假设场景图已预先完整构建，不支持在线增量构建；检索过程是开环的，无法根据规划中间结果动态调整。

6.3 EmbodiedRAG：任务驱动的场景图子图检索

Booker 等人[20]提出的 EmbodiedRAG（2024）将 RAG（Retrieval-Augmented Generation）范式适配到具身智能领域，提出 3D 场景图子图检索框架，替代 SayPlan 的手动折叠/展开机制。

EmbodiedRAG 的核心技术组件包括：（1）场景图文档索引——将场景图中每个实体节点视为一个"文档"，嵌入到向量存储中；（2）LLM 引导的预检索抽象——LLM 仅基于任务文本推断可能相关的实体类型和属性，作为向量检索的初始查询；（3）子图 Grounding——以向量检索命中的节点为锚点，提取诱导子图并过滤任务相关属性；（4）Self-Query 反馈机制——LLM 规划过程中产生的"思考"被解析为新的检索查询，动态扩展检索范围。

实验结果表明（AI2Thor 模拟环境，GPT-4o-mini）：累计 token 消耗降低 90%，平均每步规划时间减少 70%，任务成功率提升。在 Boston Dynamics Spot 四足机器人上的硬件验证中，Full-Mem 方案在大环境中因 token 过载导致严重幻觉，而 EmbodiedRAG-strict 未出现幻觉。

6.4 ReMEmbR：长时时空语义记忆

Quach 等人[21]提出的 ReMEmbR（NVIDIA, 2024）解决了机器人在长时间部署（数小时到数天）中如何构建和查询语义记忆的问题。其核心架构分为两个阶段：记忆构建阶段——机器人运行过程中，短视频片段被 VLM 生成语义描述，连同时间戳和坐标信息嵌入到向量数据库中；查询阶段——LLM Agent 通过多种工具函数（文本查询、时间查询、位置查询）迭代查询向量数据库，直到回答用户问题。

ReMEmbR 的关键设计决策在于使用向量数据库而非直接存储视频，解决了长上下文推理的效率问题。在 Nova Carter 机器人上的部署验证了该方案在真实环境中的可行性，机器人能够回答"带我去最近的电梯"等需要时空推理

的查询。

6.5 GraphRAG 与 ConceptGraphs

Edge 等人[22]提出的 GraphRAG（Microsoft, 2024）虽然面向文本领域，但其核心思想对机器人场景图检索有重要启发。GraphRAG 将传统 RAG 的"文档块检索"升级为"知识图谱 + 社区摘要检索"：从源文档中提取实体和关系构建知识图谱，对图进行社区检测生成层级化社区结构，每个社区生成 LLM 摘要。对机器人场景图的启发在于：场景图天然具有社区结构（房间即社区），可以为每个房间生成语义摘要，支持全局查询。

Gu 等人[35]提出的 ConceptGraphs（ICRA 2024）提出了开放词汇 3D 场景图的构建方法，利用 2D 基础模型（SAM、CLIP）的输出通过多视角关联融合到 3D，构建开放词汇的图结构化场景表征，可泛化到训练时未见过的语义类别。

Maggio 等人[36]提出的 Clio（MIT, 2024）从另一个角度解决信息过载问题：在场景图构建阶段就进行任务驱动的压缩。给定自然语言任务列表，Clio 使用信息瓶颈方法自动选择与任务相关的语义粒度和物体子集。Clio 在感知端压缩（构建时过滤），EmbodiedRAG 在检索端压缩（查询时过滤），两者可以组合使用。

6.6 方案对比分析

维度	SayPlan	EmbodiedRAG	ReMEmbR	GraphRAG	Clio
检索方式	LLM 手动展开/折叠	向量相似度 + 子图 Grounding	向量数据库多模态查询	知识图谱 + 社区摘要	构建时任务驱动压缩
压缩阶段	查询时	查询时	查询时	索引时 + 查询时	构建时
反馈机制	无	Self-Query 闭环	Agent 迭代查询	无	无
时间维度	无	动态适应环境变化	长时时空推理	无	实时
token 压缩率	中等	90%（论文数据）	高	高	高

6.7 小结

3D 场景图为 LLM 规划提供了结构化的环境 Grounding，EmbodiedRAG 的任务驱动检索解决了大规模环境下的上下文管理问题，ReMEmbR 的长时记忆为跨任务经验积累提供了方案。本课题设计的 ARIA（Agent with Retrieval-augmented Intelligence Architecture）三层记忆架构融合了上述方案的核心思想：工作记忆（实时状态）对应 RobotState、语义记忆（场景图 + 向量索引）参考 EmbodiedRAG 的文档索引方案、情景记忆（执行历史）参考 ReMEmbR 的长时记忆思想。

7 VLA 技术与具身智能前沿

7.1 VLA 概述

VLA（Vision-Language-Action）模型是将视觉感知、自然语言理解和动作生成统一在单一学习框架中的多模态模型。根据 ICLR 2026 VLA 研究综述，VLA 的严格定义要求使用在大规模视觉-语言数据上预训练的骨干网络，随后训练生成控制命令——互联网规模的视觉-语言预训练是 VLA 区别于普通多模态策略的核心特征。

VLA 的三大核心组件包括：视觉-语言骨干网络（通常基于大型 VLM 预训练，已具备物体识别、文本理解、空间推理能力）、动作接口（离散 token 预测、连续动作回归或扩散生成等）、多模态输入（相机图像、自然语言指令及机器人状态）。

7.2 VLA 发展历程与代表工作

RT-2（Google DeepMind, 2023）[27]是 VLA 概念的开创者，首次证明可以将网络规模的视觉-语言知识迁移到机器人控制。RT-2 基于 PaLI-X（55B）VLM 骨干，将机器人动作编码为文本 token，展示了对未见过物体和指令的泛化能力。但模型规模巨大，推理速度慢。

OpenVLA（Stanford/UC Berkeley, 2024）[28]是首个高质量开源 VLA 模型，基于 Llama 2（7B）语言模型和 DINOv2/SigLIP 双流视觉编码器，在近 100 万条机器人轨迹上微调。以 7B 参数超越 RT-2-X（55B）16.5% 的绝对任务成功率，支持 LoRA 高效微调适配新场景。OpenVLA-OFT（2025）进一步提出并行解码、Action Chunking、连续动作表示等优化，推理速度提升 25-50 倍，在 LIBERO 基准上达到 97.1% 平均成功率。

π0 系列（Physical Intelligence, 2024-2025）[29]代表了 VLA 的工业前沿。π0 基于 PaliGemma（3B VLM）+ Flow Matching 动作生成架构，实现了跨形态通用策略（叠衣服、清理桌面等），控制频率达 50Hz。π0-FAST 引入频域变换动作 token 化方案，兼顾效率和精度。π0.5 是首个具备开放世界泛化能力的 VLA，首次实现端到端学习系统在全新家庭环境中完成长时序灵巧操作。

7.3 双系统架构

2025 年出现的一个显著趋势是双系统架构，受人类认知的"快思考/慢思考"理论启发，解决了大模型推理延迟与实时控制需求的根本矛盾：

Figure AI 的 Helix（2025）[30]采用 7B VLM 作为 System 2（慢思考，7-9 Hz）负责指令解释和环境分析，高频视觉运动策略作为 System 1（快思考，200 Hz）负责实时精细运动控制。这是首个输出全身上半身连续控制的 VLA。

NVIDIA 的 GR00T N1（2025）[31]是世界首个开源人形机器人基础模型，采用 Eagle-2 VLM 的视觉-语言模块（System 2）和扩散 Transformer 模块（System 1），在仿真和真实任务中超越纯模仿学习基线。

Google DeepMind 的 Gemini Robotics（2025）分为 Robotics 1.5（VLA，直接控制机器人）和 Robotics-ER 1.5（VLM，具身推理编排器），两者配合实现"ER 负责想，Robotics 负责做"。

7.4 世界模型与 VLA 融合

2025-2026 年最重要的技术趋势是世界模型（World Model）与 VLA 的融合。世界模型学习环境的因果动力学，核心能力是预测"如果智能体执行了动作 A，环境状态会如何演变"。

WorldVLA（阿里达摩院 + 浙大, 2025）将 VLA 和世界模型统一到单一自回归框架：VLA 分支从图像观测生成后续动作，世界模型分支利用动作和视觉输入预测未来图像状态，两个分支互相增强。NVIDIA Cosmos 系列将世界模型定位为通用物理 AI 基础设施，为 VLA 训练提供合成数据生成和策略仿真评估。

行业共识正在形成：世界模型是底层基础设施（仿真 + 数据 + 物理理解），VLA 是上层策略模型（感知 + 决策 + 执行），两者结合是通往通用具身智能的主流路径。

7.5 VLA 核心挑战

当前 VLA 技术仍面临多项挑战：（1）精度不足——π0 微调后放置精度误差达 2.2cm/12.4°，远不能满足工业装配需求；（2）知识遗忘——VLM 微调为 VLA 时预训练的开放世界推理能力会退化；（3）数据瓶颈——高质量机器人操作数据仍然稀缺且昂贵；（4）仿真与真实的鸿沟——仿真中的高成功率不等于真实世界的可靠性。

7.6 小结

VLA 技术的快速发展为机器人智能化提供了端到端的解决方案，但当前仍处于快速演进阶段，尚未形成稳定的工程化部署方案。本课题的系统架构设计充分考虑了 VLA 的演进方向：通过依赖倒置和 Capability 插件化架构，导航能力可从当前的 Nav2 方案整体替换为端到端 VLA 方案（只需实现新的 NavigationCapability），Agent 核心代码零修改。这种"面向未来的架构兼容性"是本系统的重要设计特征。

8 外部 AI 模型 API 集成技术

8.1 大语言模型 API 与 Function Calling

OpenAI 提供的 GPT 系列 API[10]是当前最广泛使用的大语言模型服务之一。GPT API 支持文本补全、对话生成和函数调用（Function Calling）等功能。Function Calling 机制允许开发者定义一组可调用的函数及其参数描述，LLM 能够根据用户输入自动选择合适的函数并生成调用参数。

GPT-4[10]在复杂推理、指令遵循和多模态理解方面的能力显著提升，使其能够更准确地理解用户的自然语言指令并生成结构化的任务描述。这一能力对于机器人任务调度系统中的指令解析至关重要。本课题利用 Function Calling 机制，将机器人的 ROS 2 Action 能力注册为 LLM 可调用的工具函数，从 CapabilityRegistry 动态生成函数定义，实现能力的自动发现与调用。

8.2 API 集成的关键技术挑战

在机器人系统中集成外部 AI API 面临以下技术挑战：

实时性：外部 API 调用涉及网络通信，存在不可避免的延迟（通常 1-5 秒）。在机器人系统中，任务调度的实时性直接影响系统的响应速度和用户体验。采用异步编程模型（如 Python 的 asyncio）和请求并发处理可以有效缓解延迟问题。

可靠性：网络通信的不稳定性可能导致 API 调用失败。需要设计完善的错误处理机制，包括指数退避重试策略、超时控制和降级方案，确保系统在 API 不可用时仍能维持基本功能。

安全性：API 调用涉及敏感信息（如 API 密钥）的传输，需要采用 HTTPS 加密通信，密钥通过环境变量注入而非硬编码。

8.3 小结

大语言模型 API 的 Function Calling 机制为 AI Agent 的工具调用提供了标准化接口。本课题基于 Function Calling 构建任务解析模块，并通过异步通信、指数退避重试和降级策略优化 API 调用的实时性和可靠性。

9 总结与展望

通过对上述文献的系统梳理，可以得出以下结论：

（1）ROS 2 提供了先进的通信机制和系统架构。其 Action 通信机制的"目标-反馈-结果"三阶段模型与 AI Agent 的"指令-执行-回报"交互模式天然对应，为两者的深度集成提供了架构基础。

（2）传统任务调度方法存在根本性局限。行为树和有限状态机虽然成熟可靠，但在灵活性、复用性和易用性方面难以满足智能化机器人系统的需求。一个包含导航、巡逻、异常处理的复合任务可能需要数十个行为树节点，而基于 LLM 的方案只需一句自然语言指令。

（3）LLM 工具调用范式的演进为机器人 Agent 提供了成熟的技术基础。从 Function Calling 到 ReAct Agent Loop 再到平台级 Agent 基础设施，每一层都建立在下一层之上。Function Calling 的"函数名 + 参数"模式与 ROS 2 Action 的"Goal 类型 + 参数"模式天然匹配，这一对应关系是本课题技术路线的核心依据。

（4）面向机器人的 LLM 规划已从开环演进到闭环。SayCan 的能力锚定思想、Inner Monologue 的闭环反馈机制、EmbodiedAgent 和 VeriGraph 的反幻觉研究，共同构成了可靠机器人 LLM 规划的技术基础。

（5）3D 场景图与 RAG 检索解决了大规模环境下的上下文管理问题。SayPlan 的层级场景图表征、EmbodiedRAG 的任务驱动子图检索（token 降低 90%）、ReMEmbR 的长时语义记忆，为 LLM 规划提供了精简、相关、充分的环境上下文。

（6）VLA 技术代表了具身智能的前沿方向。从 RT-2 到 π0 系列再到双系统架构（Helix、GR00T N1），VLA 正在从研究原型走向工程化部署。世界模型与 VLA 的融合是 2025-2026 年最重要的技术趋势。

当前研究的不足之处在于：现有的 AI Agent 框架多面向通用计算任务设计，尚未针对 ROS 2 的原生通信模型进行专门适配；面向机器人的 Agent 研究虽然验证了技术可行性，但缺乏与 ROS 2 系统的深度集成方案；ROS 2 社区目前缺少一个开箱即用的、基于 LLM 的通用任务调度中间层。

本课题将在上述研究的基础上，设计并实现 MOSAIC（Modular Orchestration System for Agent-driven Intelligent Control）系统——一种面向 ROS 2 架构的 AI Agent 机器人任务调度系统。系统融合 SayCan 的能力锚定思想（通过 Function Calling 实现更轻量的方案）、EmbodiedRAG 的场景图检索（ARIA 三层记忆架构）、VeriGraph 的场景图验证（SceneGraphValidator）、Inner Monologue 的闭环反馈（执行中 LLM 重规划），并通过依赖倒置和 Capability 插件化架构预留向 VLA 端到端方案演进的扩展点，为 ROS 2 生态提供一种可复用的智能任务调度方案。

参考文献

[1] Quigley M, Conley K, Gerkey B, et al. ROS: an open-source Robot Operating System[C]//ICRA Workshop on Open Source Software, 2009.

[2] Maruyama Y, Kato S, Azumi T. Exploring the performance of ROS2[C]//International Conference on Embedded Software (EMSOFT), 2016: 1-10.

[3] Macenski S, Foote T, Gerkey B, et al. Robot Operating System 2: Design, architecture, and uses in the wild[J]. Science Robotics, 2022, 7(66): eabm6074.

[4] Koubaa A. Robot Operating System (ROS): The Complete Reference (Volume 6)[M]. Springer, 2021.

[5] Colledanchise M, Ögren P. Behavior Trees in Robotics and AI: An Introduction[M]. CRC Press, 2018.

[6] Macenski S, Martín F, White R, et al. The Marathon 2: A Navigation System[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.

[7] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. 2nd ed. Cambridge: MIT Press, 2018.

[8] Li X, Xiao L. Integrating AI and Robotics for Autonomous Task Scheduling and Execution[J]. IEEE Access, 2021, 9: 12345-12356.

[9] Russell S, Norvig P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Pearson, 2021.

[10] OpenAI. GPT-4 Technical Report[R]. arXiv preprint arXiv:2303.08774, 2023.

[11] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations (ICLR), 2023.

[12] Liang J, Huang W, Xia F, et al. Code as Policies: Language Model Programs for Embodied Control[C]//IEEE International Conference on Robotics and Automation (ICRA), 2023.

[13] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need[C]//Advances in Neural Information Processing Systems (NeurIPS), 2017: 5998-6008.

[14] Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge: MIT Press, 2016.

[15] Ahn M, Brohan A, Brown N, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances[C]//Conference on Robot Learning (CoRL), 2022.

[16] Huang W, Xia F, Xiao T, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models[C]//Conference on Robot Learning (CoRL), 2022.

[17] Dalal M, et al. Grounding LLMs For Robot Task Planning Using Closed-loop State Feedback[R]. arXiv:2402.08546, 2024.

[18] Driess D, et al. PaLM-E: An Embodied Multimodal Language Model[C]//International Conference on Machine Learning (ICML), 2023.

[19] Rana K, et al. SayPlan: Grounding Large Language Models Using 3D Scene Graphs for Scalable Robot Task Planning[C]//Conference on Robot Learning (CoRL), 2023.

[20] Booker M, et al. EmbodiedRAG: Dynamic 3D Scene Graph Retrieval for Efficient and Scalable Robot Task Planning[R]. arXiv:2410.23968, 2024.

[21] Quach J, et al. ReMEmbR: Building and Reasoning Over Long-Horizon Spatio-Temporal Memory for Robot Navigation[R]. arXiv:2409.13682, 2024.

[22] Edge D, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization[R]. arXiv:2404.16130, Microsoft Research, 2024.

[23] Wan H, et al. EmbodiedAgent: A Scalable Hierarchical Approach to Overcome Practical Challenge in Multi-Robot Control[R]. arXiv:2504.10030, 2025.

[24] Ekpo D, et al. VeriGraph: Scene Graphs for Execution Verifiable Robot Planning[R]. arXiv:2411.10446, 2024.

[25] Zeng H, et al. H-AIM: Orchestrating LLMs, PDDL, and Behavior Trees for Hierarchical Multi-Robot Planning[R]. arXiv:2601.11063, 2026.

[26] Chen Y, et al. EMOS: Embodiment-aware Heterogeneous Multi-robot Operating System with LLM Agents[C]//International Conference on Learning Representations (ICLR), 2025.

[27] Brohan A, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[R]. arXiv:2307.15818, Google DeepMind, 2023.

[28] Kim M, et al. OpenVLA: An Open-Source Vision-Language-Action Model[R]. arXiv:2406.09246, 2024.

[29] Physical Intelligence. π0: A Vision-Language-Action Flow Model for General Robot Control[R]. arXiv:2410.24164, 2024.

[30] Figure AI. Helix: A Vision-Language-Action Model for Generalist Humanoid Control[R]. 2025.

[31] NVIDIA. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots[R]. arXiv:2503.14734, 2025.

[32] OpenAI. API Documentation[EB/OL]. https://platform.openai.com/docs/.

[33] Liu N F, et al. Lost in the Middle: How Language Models Use Long Contexts[J]. Transactions of the Association for Computational Linguistics (TACL), 2024.

[34] Shi F, et al. Large Language Models Can Be Easily Distracted by Irrelevant Context[C]//International Conference on Machine Learning (ICML), 2023.

[35] Gu Q, et al. ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning[C]//IEEE International Conference on Robotics and Automation (ICRA), 2024.

[36] Maggio D, et al. Clio: Real-time Task-Driven Open-Set 3D Scene Graphs[J]. IEEE Robotics and Automation Letters (RA-L), 2024.